

DEMO: Original synthetic editorial-evaluation material. All numerical results are authored fixtures, not measurements from real experiments or a publication.

Staging Base-Referenced Tile Patches for Coherent Raster Updates

Abstract

A dashboard can receive changed tiles quickly while briefly showing a raster that the producer never published. Staging a complete snapshot already prevents this mixture, but transfers unchanged data. We describe patches that name a base revision, reuse its unchanged tiles, and publish replacements together after validation. This extends an existing immutable-root renderer: dirty-tile tracking and atomic publication are inherited capabilities. In fixed synthetic fixtures, patch staging reduces median completion delay from 87 to 17 ms for four changed tiles on a constrained link, with no mixed root at any prescribed reader interleaving. Yet it is slower than full snapshots for dense changes and for sparse changes on a fast local link. The contribution is a concrete protocol for transferring less while preserving coherent display, together with evidence that its added work is worthwhile only under some of the illustrated conditions.

1. Transfer Less Without Showing an Unpublished Raster

A dashboard displays a committed raster revision, not simply the newest tile received at each coordinate. Suppose revision 10 contains tile versions a0 and b0, and revision 11 needs a1 and b1. Installing a1 while b1 is delayed exposes (a1, b0), which belongs to neither revision. Revision tags identify the intended update but do not prevent this mixture. This two-tile sequence illustrates a failure mode within the 64-tile system; it adds no timing observation.

The existing renderer already captures one published root at the start of each frame. Its root contains 64 pointers to immutable tile versions, each with 4,096 four-byte cells; a full payload is 1,048,576 bytes. Root copying, reference counting, sparse invalidation, checksums, and atomic local publication predate the transfer changes. Raster values are assumed computed; their environmental accuracy is not evaluated.

Staging a full snapshot already extends coherent publication to network delivery. The remaining question is whether changed-tile transfer can preserve that behavior at lower cost. A staged patch answers by naming the base whose unchanged tiles may be reused, assembling replacements privately, and publishing only a validated target. Maintaining that relationship through delayed packets and reconnection requires more than tagging each tile. The comparisons below assess this extra work against full snapshots, including conditions where transmitting fewer bytes does not finish sooner.

2. Constructing and Publishing a Target

Full snapshot (FS) stages an entire raster, validates all packets, and switches the visible root. Immediate tiles (IT) sends changed tiles with a target revision tag and installs each pointer on arrival. Staged patch (SP) instead supplies a manifest naming the base, target, and changed-tile set. The client pins its visible base and builds a private pending root from unchanged base tiles and validated replacements. FS and SP both use the pre-existing root swap.

In the two-tile example, SP keeps (a0, b0) visible while a1 is pending; after both replacements validate, the next reader sees (a1, b1). An unchanged tile can remain shared from the base. Coherence means matching the producer's target root, not giving every tile a new timestamp: revision 11 can legitimately include a tile created in revision 8. Readers do not wait for pending packets, although they may display an older coherent revision. This does not imply that all clients show the same revision simultaneously.

The client acknowledges only the root it has published. The server derives a patch from that acknowledged revision when available, retaining a chain of manifests for at most eight completed revisions. Coalescing takes the union of touched coordinates across the chain and sends their values from the target root, once per coordinate. An earlier intermediate value for a subsequently changed tile would not reconstruct the target. A tile that changes and then returns to its original bytes may still be sent; this is not a minimal-difference compressor.

The pending root remains private until all declared replacements validate and the commit is checked against the pinned base. Reordered packets may be staged under their target; matching duplicates are idempotent, while conflicting duplicates reject the patch. Size, checksum, and tile-range validation reject malformed data. Invalid or incomplete pending patches are discarded, leaving the last coherent revision visible. Disconnect also discards uncommitted state; late valid packets must not modify a newer visible root. If the acknowledged base is unavailable, incorrect, or unverifiable, the client requests FS. These rules concern volatile display state, not durable transactions or process-crash recovery.

Retaining the base has a cost. SP holds old tiles and replacements until old readers release their roots; FS similarly holds an old visible frame and a new full frame. Each pending update also needs a 512-byte pointer table and 4,096 bytes of descriptors and reference counters. Dense SP updates may require a full frame of pending payload. No bound on old-reader pinning lifetime has been established.

3. What the Fixed Comparisons Show

The synthetic fixtures use 120 target roots per row, identical lossless tile values, and an identical packet schedule within each workload/link pair. Completion delay extends from producer target publication until every required packet has been applied or staged; it is not first-byte latency. A coherent run has no non-target mixed root at any prescribed reader interleaving from request through completion. This finite criterion permits an older coherent display and does not establish correctness for all schedules.

Three patterns change four sparse tiles, 24 tiles in a moving front, or 60 dense tiles. Only the sparse pattern appears on the fast local link. All rows fix queue capacity at 32 packets and the retry deadline at 20 ms; an older 64-packet exploratory run is excluded. No compression is enabled. Serialization and transport are identical except for necessary payload selection and manifest handling. Each case has one fixed schedule, with no repeated trials, confidence intervals, stochastic loss model, or CPU microarchitecture study.

Table 1. Coherence-preserving comparisons: fixed DEMO values for FS and SP. Delay and preparation are milliseconds; wire sizes include protocol framing. KiB and MiB are binary units. Retention counts payload only, under the reader assumption discussed below.

Pattern / link	Method	Coherent runs	Median delay	p95 delay	Median wire KiB	Peak payload MiB	Median prepare
Sparse 4 / constrained	FS	120/120	87	106	1026	2.00	0.18
Sparse 4 / constrained	SP	120/120	17	31	68	1.06	0.32
Moving 24 / constrained	FS	120/120	89	110	1026	2.00	0.18
Moving 24 / constrained	SP	120/120	51	74	388	1.38	0.37
Dense 60 / constrained	FS	120/120	91	111	1026	2.00	0.18
Dense 60 / constrained	SP	120/120	97	120	964	1.94	0.44
Sparse 4 / fast local	FS	120/120	2.2	3.1	1026	2.00	0.18
Sparse 4 / fast local	SP	120/120	2.6	3.9	68	1.06	0.32

FS already meets the coherence criterion in every case, as does SP. With four and 24 changed tiles on the constrained link, SP reduces median completion from 87 to 17 ms and from 89 to 51 ms, respectively. Wire sizes fall from 1,026 KiB to 68 and 388 KiB, and p95 delays also improve. For 60 changed tiles, however, fewer transmitted bytes accompany worse median and p95 completion: 91 to 97 ms and 111 to 120 ms. The fast-link sparse case also reverses the delay advantage, from 2.2 to 2.6 ms at the median and 3.1 to 3.9 ms at p95.

SP preparation is higher in every pair, at 0.32–0.44 ms versus 0.18 ms for FS. As constrained-link changes become denser, SP retained payload grows from 1.06 through 1.38 to 1.94 MiB, approaching FS's 2.00 MiB. These retention values exclude pointer tables and descriptors and assume one reader finishes within one measured display cycle; they are not total-memory bounds for stalled readers.

Table 2. Diagnostic IT results, retaining every supplied metric. Units and delay definitions match Table 1. Lower delay here does not establish a coherence-preserving advantage.

Pattern / link	Method	Coherent runs	Median delay	p95 delay	Median wire KiB	Peak payload MiB	Median prepare
Sparse 4 / constrained	IT	71/120	13	26	66	1.06	0.05
Moving 24 / constrained	IT	40/120	47	69	386	1.38	0.07
Dense 60 / constrained	IT	31/120	83	104	962	1.94	0.09
Sparse 4 / fast local	IT	71/120	1.3	2.2	66	1.06	0.05

IT's failures show why eventual packet application is insufficient: a reader may already have seen a mixed root. Separate functional fixtures address interruption handling. SP matches the target-root oracle in all 28 hand-authored traces, covering first connection without a base, empty patches, delayed final tiles, reordered packets, matching and conflicting duplicates, corruption, disconnect during staging, late packets after reconnect, base eviction, and two pinned readers. Eight base-eviction/reconnect traces take FS fallback; there is no recovery-latency summary or demonstrated sparse-path byte saving on fallback.

The development variants expose necessary conditions. Removing base checking produces mismatched roots in two of six reconnect traces; acknowledging the first packet instead of publication fails after coalescing in one of six. Target-value coalescing and malformed-tile-ID rejection are also functionally checked. These results support the protocol decisions without serving as timing ablations or exhaustive proofs.

4. Discussion: When the Added Work Is Worthwhile

The useful distinction is conditional efficiency among coherent designs. The sparse and moving-front constrained-link fixtures favor patches, while the dense and fast-link fixtures favor snapshots. Higher preparation cost is a plausible contributor to the reversals, but these comparisons do not isolate manifest handling or establish that latency differences arise solely from byte count. They provide no universal crossover threshold or evaluated adaptive selector.

Base availability also limits the useful patch path. Fallback preserves a coherent display but changes the transfer cost; the 28 functional traces must remain separate from the completion table. Allocation-failure injection and process-crash testing are unimplemented. Client-server security, authentication, and Byzantine behavior are outside scope. The sources include no literature survey or deployment evidence, so this demonstration establishes neither first-of-kind novelty nor production readiness.

5. Conclusion

Naming and pinning the published base lets changed tiles be assembled into a coherent target before exposure. Acknowledgement after publication and explicit fallback keep that relationship meaningful across interruptions.

ORIGINAL DEMO | SYNTHETIC VALUES | NOT EXPERIMENTAL RESULTS

In the synthetic fixtures, the design preserves the coherence already provided by full snapshots and improves completion only in some transfer conditions. The result is a bounded protocol and cost argument, not measured real-world performance.